

Stage 2 Step-by-Step Practical & Conceptual Recap

1. Defining the In-Memory Data Store (RAM Cache)

- **What You Did:** In `server.js`, you created an array of 3 task objects in volatile memory using the schema `id` (number), `title` (text), and `done` (boolean):
 - Task 1: *"Responding to routine client emails"* (`done: false`)
 - Task 2: *"Monitoring server status alerts"* (`done: false`)
 - Task 3: *"Prepare for client meeting"* (`done: false`)
- **Sysadmin Analogy:** This is like loading a lightweight lookup table directly into active system RAM or a Windows Registry cache. If the Node process stops or restarts, the memory resets to its initial seeded state—exactly like volatile system cache clearing on a server reboot.

2. Reading All Records (GET /tasks)

- **What You Did:** You created the route `app.get('/tasks', (req, res) => res.status(200).json(tasks));`.
- **Sysadmin Analogy:** This is the equivalent of running `Get-Service` or `Get-Process` in PowerShell. It queries the active system state and dumps the full inventory list to the requesting client.

3. Dynamic Path Parameters (GET /tasks/:id)

- **What You Did:** You created a dynamic route using Express route parameters `(:id)` to capture dynamic values from the URL path.
- **Key Code Logic:**
 1. `const taskId = parseInt(req.params.id, 10);` — Converts the incoming URI string parameter into a base-10 integer.
 2. `const task = tasks.find(t => t.id === taskId);` — Searches the array memory space for a matching primary key.
- **Sysadmin Analogy:** The colon prefix `(:id)` works like a positional argument or variable parameter in a PowerShell script (e.g., `Get-WinEvent -InstanceId $EventID`). The router extracts whatever value is passed in the URL string and passes it into your lookup query.

4. Guard Clauses & Defensive HTTP 404 Logic

- **What You Did:** You added a firewall-style validation gate:

```
if (!task) {  
  return res.status(404).json({ error: `Task ${taskId} not found` });  
}
```

- **Why This Matters:** Machines, automated monitoring systems, and API client scripts depend on HTTP status codes to determine success or failure. Returning an empty 200 OK when data doesn't exist misleads downstream systems.
- **Sysadmin Analogy:** When querying an event log for a non-existent Record ID, a robust system generates a clear error response (404 Not Found) rather than returning a blank success response.

5. Execution & Daemon Process Control

- **What You Did:** You saved server.js (Ctrl + S), went to your **left VS Code terminal tab**, stopped any previous process (Ctrl + C), and started the daemon:
node server.js
- **Observed Daemon Output:**
Task API Daemon running on http://localhost:3000
- **Sysadmin Analogy:** Starting node server.js binds the Express server process to TCP Port 3000 as a foreground service daemon, waiting to intercept inbound TCP requests.

6. Protocol Verification (curl.exe Inspection)

- **What You Did:** In your **right VS Code terminal tab**, you tested both success and failure HTTP routes using curl.exe:
 1. **Success Test (Existing ID):**
curl.exe -i http://localhost:3000/tasks/1
 - **Response Header:** HTTP/1.1 200 OK
 - **Payload Body:** {"id":1,"title":"Responding to routine client emails","done":false}
 2. **404 Error Test (Non-Existent ID):**
curl.exe -i http://localhost:3000/tasks/99
 - **Response Header:** HTTP/1.1 404 Not Found
 - **Payload Body:** {"error":"Task 99 not found"}
- **Sysadmin Analogy:** Running curl.exe with -i inspects raw response headers and status codes, acting like a network protocol analyzer (similar to Wireshark or Test-NetConnection) to confirm that your API communication layer adheres strictly to HTTP standards.

7. Version Control Checkpoint & Commit

- **What You Did:** You staged all changes and created a restore point in Git:
git add .
git commit -m "stage2: read endpoints with 404"
- **Observed Audit Log Output:** [master 7c42061] stage2: read endpoints with 404
- **Sysadmin Analogy:** Creating a Git commit creates an immutable snapshot of your code

base before you write creation (POST) and update (PUT/PATCH) endpoints in Stage 3.